

Clever Specification Feedback

We would like to share some observations regarding the specifications in the Clever benchmark, and we hope this feedback may be helpful for further improving the benchmark. We would like to emphasize that our intent is entirely constructive, and we greatly appreciate the effort that has gone into building Clever.

Our primary focus is on the *strength* of the specifications, that is, whether each postcondition is precise enough to characterize the intended behavior and rule out unintended outputs. In the course of this investigation, we also came across a small number of issues in the provided implementations and test cases, which we include alongside the specification observations for completeness. We believe that the precision of specifications is important, because it determines whether proofs carry meaningful correctness guarantees. As an illustrative example, if a postcondition happens to be vacuously satisfiable, then any Lean program that passes type checking would verify successfully, without actually establishing the intended property.

In practice, most postconditions in Clever are far from vacuously true, and even specifications with minor gaps can still provide valuable correctness guarantees for individual functions. Nevertheless, subtle underspecification may become problematic when specifications are composed in larger verification efforts, where higher-level proofs rely on postconditions as precise contracts. As a concrete example, consider Problem 79 (`decimal_to_binary`), where the expected output for input 0 is `"db0db"`. One might assume the postcondition uniquely identifies this result, but we observed that `"db/db"` also satisfies it. The reason is that the specification uses `fun c => c.toNat - '0'.toNat` to convert each trimmed character to a digit value. In Lean 4, subtraction on `Nat` is truncated, so if the result would be negative, it is clamped to 0. Since `'/'` has ASCII code 47, one less than `'0'` (ASCII 48), `'/' .toNat - '0'.toNat` evaluates to 0, the same value as the digit `'0'`, allowing `"db/db"` to satisfy the postcondition.

In this report, we have identified 18 potential issues across the Clever benchmark. We hope the following overview and detailed analysis are useful.

The observations in this report are based on version 1.4.0 of the Clever benchmark (commit 603e318, the latest available at the time of writing).

Problem ID	Potential Issue
Problem 7	Underspecified Post-condition
Problem 9	Implementation Issue
Problem 24	Underspecified Post-condition
Problem 27	Underspecified Post-condition
Problem 29	Underspecified Post-condition
Problem 30	Underspecified Post-condition
Problem 44	Underspecified Post-condition
Problem 49	Underspecified Post-condition
	Possible Test Case Discrepancy
Problem 68	Underspecified Post-condition
Problem 69	Underspecified Post-condition
Problem 79	Underspecified Post-condition
Problem 84	Underspecified Post-condition
Problem 88	Underspecified Post-condition
Problem 89	Underspecified Post-condition
Problem 94	Possible Incorrect Specification
Problem 96	Underspecified Post-condition
Problem 156	Underspecified Post-condition
Problem 161	Underspecified Post-condition

Problem 7

Problem Description

Description

```
def filter_by_substring(strings: List[str], substring: str) -> List[str]:
    """Filter an input list of strings only for ones that contain given
    ↪ substring."""
    ...
```

Test Cases

```
filter_by_substring([], "a") # => []
filter_by_substring(["abc", "bacd", "cde", "array"], "a") # => ["abc", "bacd",
↪ "array"]
```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: List String → String → List String)
-- inputs
(strings: List String)
(substring: String)
:=
-- spec
let spec (result: List String) :=
(∀ i, i < result.length → result[i]!.containsSubstr substring →
```

```

 $\forall j, j < \text{strings.length} \wedge \text{strings}[j]!. \text{containsSubstr substring} \rightarrow \text{strings}[j]! \in$ 
 $\hookrightarrow \text{result} \rightarrow$ 
 $\forall j, j < \text{result.length} \rightarrow \text{result.count result}[j]! = \text{strings.count result}[j]!);$ 
-- program termination
 $\exists \text{result}, \text{implementation strings substring} = \text{result} \wedge$ 
spec result

```

Observation

The post-condition appears to be underspecified.

A potential concern is that the three intended conditions, namely (1) every element in `result` contains the substring; (2) every element in `strings` that contains the substring appears in `result`; and (3) the multiplicity of each element in `result` matches that in `strings`, are written as a single nested implication chain rather than three independent conjuncts. As a result, conditions (1) and (2) serve only as antecedents for condition (3), and the specification may be vacuously satisfied whenever any of these antecedents is false.

In particular, returning an empty list always satisfies the specification, since the outermost quantifier $\forall i, i < \text{result.length} \rightarrow \dots$ has a false antecedent when `result` is empty. For example:

```

strings = ["abc", "bacd", "array"]
substring = "a"
result = [] # vacuously satisfies the spec

```

even though the correct output is ["abc", "bacd", "array"].

Problem 9

Problem Description

Description

```

def rolling_max(numbers: List[int]) -> List[int]:
    """From a given list of integers, generate a list of rolling maximum element
     $\hookrightarrow$  found until given
    moment in the sequence."""
    ...

```

Test Cases

```

rolling_max([1, 2, 3, 2, 3, 4, 2]) # => [1, 2, 3, 3, 3, 4, 4]

```

Clever's Spec

```

def problem_spec
-- function signature
(implementation: List Int  $\rightarrow$  List Int)
-- inputs
(numbers: List Int) :=
-- spec
let spec (result: List Int) :=
result.length = numbers.length  $\wedge$ 

```

```

 $\forall i, i < \text{numbers.length} \rightarrow$ 
 $(\text{result}[i]! \in \text{numbers.take } (i + 1) \wedge$ 
 $\forall j, j \leq i \rightarrow \text{numbers}[j]! \leq \text{result}[i]!);$ 
-- program termination
 $\exists \text{result}, \text{implementation numbers} = \text{result} \wedge$ 
spec result

```

Clever's Implementation

```

def implementation (numbers: List Int) : List Int :=
-- end_def implementation_signature
-- start_def implementation
let rec rolling_max (numbers: List Int) (results: List Int) (acc: Int) : List Int
 $\hookrightarrow$  :=
  match numbers with
  | [] => results
  | n :: ns =>
    let new_acc := max acc n
    let new_results := results ++ [new_acc]
    rolling_max ns new_results new_acc
rolling_max numbers [] 0

```

Observation

The implementation appears to produce unintended results for inputs containing negative numbers.

A potential concern is that the helper function `rolling_max` is initialized with `acc := 0`, meaning the running maximum starts at 0 rather than at the first element of the list. As a result, when all elements are negative, the first entry of the result may be 0 instead of the intended prefix maximum. For example:

```

numbers = [-3, -1, -2]
expected = [-3, -1, -1] # intended rolling max
result   = [0, 0, 0]    # produced by the implementation (acc starts at 0)

```

A possible remedy would be to initialize `acc` with the first element of the list rather than a hard-coded 0, treating the empty list as a separate base case.

Problem 24

Problem Description

Description

```

def largest_divisor(n: int) -> int:
    """For a given number n, find the largest number that divides n evenly,
     $\hookrightarrow$  smaller than n."""
    ...

```

Test Cases

```

largest_divisor(15) # => 5

```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: Nat → Nat)
-- inputs
(n: Nat) :=
-- spec
let spec (result: Nat) :=
0 < n → 0 < result → result | n → ∀ x, x | n → x ≠ n → x ≤ result;
-- program termination
∃ result, implementation n = result ∧
spec result
```

Observation

The post-condition appears to be underspecified.

A potential concern is that the conditions $0 < \text{result}$ and $\text{result} \mid n$ appear as antecedents in the implication chain rather than as independent conjunctive constraints on **result**. As a result, the specification may be vacuously satisfied whenever either antecedent is false. In particular, $\text{result} = 0$ always satisfies the specification since $0 < \text{result}$ is false.

Furthermore, the specification does not require $\text{result} < n$, so $\text{result} = n$ may also be accepted: since $n \mid n$, the antecedents all hold, and the conclusion $\forall x, x \mid n \rightarrow x \neq n \rightarrow x \leq \text{result}$ reduces to $x \leq n$, which is trivially true for all divisors. For example:

```
n = 15
result = 0   # vacuously satisfies the spec (0 < result is false)
result = 15  # also satisfies the spec (all divisors of 15 are ≤ 15)
```

even though only 5 reflects the intended behavior.

Problem 27

Problem Description

Description

```
def flip_case(string: str) -> str:
    """For a given string, flip lowercase characters to uppercase and uppercase
    ↪ to lowercase."""
    ...
```

Test Cases

```
flip_case("Hello") # => "hELLO"
```

Clever's spec

```
def problem_spec
-- function signature
(implementation: String → String)
-- inputs
```

```

(string: String) :=
  -- spec
  let spec (result: String) :=
    let chars_in_result := result.toList;
    let chars_in_string := string.toList;
    chars_in_result.length = string.length ∧
    (∀ i, i < chars_in_result.length →
      let c := chars_in_result.get! i;
      let c' := chars_in_string.get! i;
      (c.isUpper → c'.isLower) ∧
      (c.isLower → c'.isUpper) ∧
      ((¬ c.isUpper ∧ ¬ c.isLower) → c = c'))
    );
  -- program termination
  ∃ result, implementation string = result ∧
  spec result

```

Observation

The post-condition appears to be underspecified.

The specification constrains the output in three ways: (1) if a character was lowercase before the transformation, it must be uppercase afterwards; (2) if a character was uppercase before, it must be lowercase afterwards; (3) if a character was not a letter, it must remain unchanged. However, the specification does not require that each letter is mapped to its *corresponding* character under case-flipping. As a result, any uppercase letter could be returned in place of a lowercase input, regardless of which letter it is. As a counterexample:

```

string = "a"
result = "A" # satisfies the post-condition
result' = "B" # also satisfies the post-condition

```

even though only "A" reflects the intended behavior.

Problem 29

Problem Description

Description

```

def filter_by_prefix(strings: List[str], prefix: str) -> List[str]:
    """Filter an input list of strings only for ones that start with a given
    ↪ prefix."""
    ...

```

Test Cases

```

filter_by_prefix([], "a") # => []
filter_by_prefix(["abc", "bcd", "cde", "array"], "a") # => ["abc", "array"]

```

Clever's Spec

```

def problem_spec
  -- function signature

```

```

(implementation: List String → String → List String)
-- inputs
(strings: List String)
(pref: String) :=
-- spec
let spec (result: List String) :=
result.all (λ s => s.startsWith pref) ∧
result.all (λ s => s ∈ strings) ∧
strings.all (λ s => s.startsWith pref → s ∈ result) ∧
∀ s : String, s ∈ result → result.count s = strings.count s;
-- program termination
∃ result, implementation strings pref = result ∧
spec result

```

Observation

The post-condition appears to be underspecified.

A potential concern is that the specification does not enforce the relative order of elements in `result`. The four conjuncts collectively ensure that `result` contains exactly the elements of `strings` that start with `pref`, with matching multiplicities, but no constraint is placed on their ordering. As a result, any permutation of the filtered elements would satisfy the specification. For example:

```

strings = ["abc", "bcd", "array"]
pref    = "a"
result  = ["abc", "array"] # satisfies the spec (original order preserved)
result' = ["array", "abc"] # also satisfies the spec (order permuted)

```

even though the intended behavior is to return the matching elements in their original order. We note that if a verifier's own specification additionally requires order preservation, it would be strictly stronger than the provided specification, which could cause a conforming implementation to fail verification against that stricter spec. We therefore suggest adding an explicit ordering constraint, such as requiring `result` to be a subsequence of `strings`.

Problem 30

Problem Description

Description

```

def get_positive(l: list):
    """Return only positive numbers in the list."""
    ...

```

Test Cases

```

get_positive([-1, 2, -4, 5, 6]) # => [2, 5, 6]
get_positive([5, 3, -5, 2, -3, 3, 9, 0, 123, 1, -10]) # => [5, 3, 2, 3, 9, 123,
↪ 1]

```

Clever's Spec

```

def problem_spec

```

```

-- function signature
(implementation: List Int → List Int)
-- inputs
(numbers: List Int) :=
-- spec
let spec (result: List Int) :=
  result.all (λ x => x > 0 ∧ x ∈ numbers) ∧
  numbers.all (λ x => x > 0 → x ∈ result) ∧
  result.all (λ x => result.count x = numbers.count x);
-- program termination
∃ result,
  implementation numbers = result ∧
  spec result

```

Observation

The post-condition appears to be underspecified.

A potential concern is that the specification does not enforce the relative order of elements in **result**. The three conjuncts collectively ensure that **result** contains exactly the positive elements of **numbers** with matching multiplicities, but no constraint is placed on their ordering. As a result, any permutation of the positive elements would satisfy the specification. For example:

```

numbers = [5, 3, 2]
result  = [5, 3, 2]  # satisfies the spec (original order preserved)
result' = [2, 5, 3]  # also satisfies the spec (order permuted)

```

even though the intended behavior is to return positive elements in their original order. As with Problem 29, a verifier whose specification additionally requires order preservation would be strictly stronger than the provided specification, potentially causing a conforming implementation to fail. We therefore suggest adding an explicit ordering constraint, such as requiring **result** to be a subsequence of **numbers**.

Problem 44

Problem Description

Description

```

def change_base(x: Nat, base: Nat) -> String:
  """Change numerical base of input number x to base.
  Return string representation after the conversion.
  Base numbers are less than 10."""
  ...

```

Test Cases

```

change_base(8, 3) # => "22"
change_base(8, 2) # => "1000"
change_base(7, 2) # => "111"

```


Clever's Spec

```
def problem_spec
-- function signature
(implementation: Nat → Nat → String)
-- inputs
(x base: Nat) :=
-- spec
let spec (result: String) :=
let result_array := result.toList.map (fun c => c.toNat - '0'.toNat);
let pow_array := (List.range result_array.length).map (fun i =>
  ↪ base^(result_array.length - i - 1) * result_array[i]!);
let pow_sum := pow_array.sum;
(0 < base ∧ base ≤ 10) ∧
(∀ i, i < result_array.length →
result_array[i]! < base ∧ 0 ≤ result_array[i]! →
pow_sum = x);
-- program termination
∃ result, implementation x base = result ∧
spec result
```

Observation

The post-condition appears to be underspecified.

The inner implication $\text{result_array}[i]! < \text{base} \wedge 0 \leq \text{result_array}[i]! \rightarrow \text{pow_sum} = x$ places the equality $\text{pow_sum} = x$ as the *conclusion* of an implication whose premise is that each digit is in range. This means the spec only requires the positional sum to equal x when all digits happen to be valid; it does not independently require that the digits are valid. As a result, an implementation can return a string containing out-of-range digits and still satisfy the specification, because the implication is vacuously false for those digits.

The intended condition should separate the two requirements: digit validity as a universally quantified conjunct, and the positional sum equality as an independent conjunct outside the quantifier:

```
(∀ i, i < result_array.length →
  result_array[i]! < base ∧ 0 ≤ result_array[i]!) ∧
pow_sum = x
```

For example, with $x = 8$ and $\text{base} = 3$, the string "30" (whose digit 3 is out of range for base 3) vacuously satisfies the current spec because the premise $3 < 3$ is false, so the implication holds trivially.

Problem 49

Problem Description

Description

```
def modp(n: Nat, p: Nat) → Nat:
  """Return 2^n modulo p (be aware of numerics)."""
  ...
```

Test Cases

```
modp(3, 5)      # => 3
modp(1101, 101) # => 2
modp(0, 101)    # => 0
modp(100, 101)  # => 1
```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: Nat → Nat → Nat)
-- inputs
(n p: Nat) :=
-- spec
let spec (result: Nat) :=
0 < p ∧
result < p ∧
(∃ k : Nat, p * k + result = Nat.pow 2 n)
-- program termination
∃ result, implementation n p = result ∧
spec result
```

Observation

Two potential issues are identified.

Issue 1: possible misplacement of precondition as a conjunct.

The spec opens with $0 < p \wedge \text{result} < p \wedge \dots$, which places $0 < p$ as a conjunct that the result must establish. However, $0 < p$ is a property of the *input* rather than of the output, and may be better expressed as a precondition connected by implication:

```
0 < p →
result < p ∧
(∃ k : Nat, p * k + result = Nat.pow 2 n)
```

With the current $\wedge$, the specification appears to require the implementation to *produce* a proof that $0 < p$, which may not reflect the intended meaning.

Issue 2: possible discrepancy in test case 3.

For the input $n = 0$, $p = 101$, we have $2^0 = 1$, so the unique solution satisfying $101k + \text{result} = 1$ with $\text{result} < 101$ is $k = 0$, $\text{result} = 1$. However, the test case provides an expected output of 0:

```
modp(0, 101) # expected_output: 0 (possibly incorrect; may should be 1)
```

Since $101 \cdot k + 0 = 1$ has no solution in the natural numbers, $\text{result} = 0$ does not appear to satisfy the specification. The expected output may be 1.

Problem 68

Problem Description

Description

```
def pluck(numbers: List[Int]) -> List[Int]:  
  """Given an array representing a branch of a tree that has non-negative  
  ↪ integer nodes  
  your task is to pluck one of the nodes and return it.  
  The plucked node should be the node with the smallest even value.  
  If multiple nodes with the same smallest even value are found return the node  
  ↪ that has smallest index.  
  
  The plucked node should be returned in a list, [ smallest_value, its index ],  
  If there are no even values or the given array is empty, return []."""  
  ...
```

Test Cases

```
pluck([4, 2, 3])      # => [2, 1]  
pluck([1, 2, 3])      # => [2, 1]  
pluck([])             # => []  
pluck([5, 0, 3, 0, 4, 2]) # => [0, 1]
```

Clever's Spec

```
def problem_spec  
  -- function signature  
  (implementation: List Nat → List Nat)  
  -- inputs  
  (numbers: List Nat) :=  
  -- spec  
  let spec (result: List Nat) :=  
    (result.length = 0 ↔ ∀ i, i < numbers.length → numbers[i]! % 2 = 1) ∧  
    (result.length = 2 ↔ ∃ i, i < numbers.length ∧  
      numbers[i]! % 2 = 0 ∧  
      result[0]! = numbers[i]! ∧  
      result[1]! = i ∧  
      (∀ j, j < numbers.length → j < i → (numbers[j]! % 2 = 1 ∨ numbers[i]! <  
        ↪ numbers[j]!)) ∧  
      (∀ k, k < numbers.length → numbers[k]! % 2 = 0 → numbers[i]! ≤  
        ↪ numbers[k]!));  
  -- program termination  
  ∃ result, implementation numbers = result ∧  
  spec result
```

Observation

The post-condition appears to be underspecified.

A potential concern is that the specification does not explicitly constrain `result.length` to be either 0 or 2. The two biconditionals characterize the cases `result.length = 0` and `result.length = 2`, but a result of any other length may satisfy both biconditionals vacuously — specifically, when both right-hand sides happen to be false simultaneously. This can occur when the input contains

```
numbers = [4, 2, 3]
result  = [2, 1]      # intended output
result' = [10, 5, 5]  # also satisfies the spec
```

Problem 69

Description

```
def search(numbers: List[int]) -> int:
    """You are given a non-empty list of positive integers. Return the greatest
    ↪ integer that is
    ↪ greater than zero, and has a frequency greater than or equal to the value of
    ↪ the integer
    ↪ itself. The frequency of an integer is the number of times it appears in the
    ↪ list.
    ↪ If no such a value exist, return -1."""
    ...
```

```
search([4, 1, 2, 2, 3, 1])      # => 2
search([1, 2, 2, 3, 3, 3, 4, 4, 4]) # => 3
search([5, 5, 4, 4, 4])        # => -1
```

```
def problem_spec
-- function signature
(implementation: List Int → Int)
-- inputs
(numbers: List Int) :=
-- spec
let spec (result: Int) :=
0 < numbers.length ∧ numbers.all (fun n => 0 < n) →
(result ≠ -1 ↔ ∃ i : Nat, i < numbers.length ∧
  numbers[i]! = result ∧ numbers[i]! > 0 ∧
  numbers[i]! ≤ (numbers.filter (fun x => x = numbers[i]!)).length ∧
  (¬∃ j : Nat, j < numbers.length ∧
    numbers[i]! < numbers[j]! ∧ numbers[j]! ≤ numbers.count numbers[j]!));
-- program termination
∃ result, implementation numbers = result ∧
spec result
```

Observation

The post-condition appears to be underspecified.

A potential concern is that `result = -1` may always satisfy the specification whenever the input contains no element equal to `-1`. The specification uses the biconditional `result  $\neq$  -1  $\leftrightarrow \exists i, \text{numbers}[i]! = \text{result} \wedge \dots$` . When `result = -1`, the left-hand side is false. The right-hand side is also false, because the existential requires `numbers[i]! = result = -1`, which cannot hold as long as `-1` does not appear in the input. Therefore, the biconditional is vacuously satisfied for `result = -1` regardless of the input. For example:

```
numbers = [4, 1, 2, 2, 3, 1]
expected = 2      # intended output
result    = -1    # also satisfies the spec (both sides of the biconditional are
                   $\leftrightarrow$  false)
```

This is a particularly subtle gap, as the biconditional may give the impression that both directions are properly constrained. We suggest restructuring the specification so that the `result = -1` case and the positive-result case are expressed as independent conjuncts, rather than relying on a single biconditional that can be vacuously satisfied.

Problem 79

Problem Description

Description

```
def decimal_to_binary(decimal: nat) -> string:
    """You will be given a number in decimal form and your task is to convert it
     $\leftrightarrow$  to
    binary format. The function should return a string, with each character
     $\leftrightarrow$  representing
    a binary number. Each character in the string will be '0' or '1'.
    There will be an extra couple of characters 'db' at the beginning and at the
     $\leftrightarrow$  end of the string.
    The extra characters are there to help with the format."""
    ...
```

Test Cases

```
decimal_to_binary(0)    # => "db0db"
decimal_to_binary(15)   # => "db1111db"
decimal_to_binary(32)   # => "db10000db"
decimal_to_binary(103)  # => "db1100111db"
```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: Nat  $\rightarrow$  String)
-- inputs
(decimal: Nat) :=
-- spec
```

```

let spec (result: String) :=
  4 < result.length ∧
  result.drop (result.length - 2) = "db" ∧
  result.take 2 = "db" ∧
  let resultTrimmed := (result.toList.drop 2).dropLast.dropLast.map (fun c =>
    ↪ c.toNat - '0'.toNat)
  decimal = Nat.ofDigits 2 resultTrimmed.reverse
-- program termination
∃ result, implementation decimal = result ∧
spec result

```

Observation

The post-condition appears to be underspecified.

A potential concern arises from the use of `fun c => c.toNat - '0'.toNat` to convert each trimmed character to a digit value. In Lean 4, subtraction on `Nat` is truncated: if the result would be negative, it is clamped to 0. Therefore, any character whose ASCII code is less than or equal to that of `'0'` (ASCII 48) — such as `'/'` (ASCII 47) — will be silently mapped to 0, indistinguishable from the digit `'0'`. As a result, strings containing such characters may satisfy the specification even though they are not valid binary representations. For example, for the input `decimal = 0`:

```

decimal = 0
expected = "db0db" # intended output
result = "db/db" # also satisfies the spec

```

For `result = "db/db"`: the trimmed portion is `['/']`, and `'/'`.toNat - `'0'`.toNat = 47 - 48 = 0 (by `Nat` truncation), so `Nat.ofDigits 2 [0] = 0 = decimal` (✓). We suggest adding an explicit validity check requiring each trimmed character to be `'0'` or `'1'`.

Problem 84

Problem Description

Description

```

def solve(n: int) -> str:
  """Given a positive integer N, return the total sum of its digits in
  ↪ binary."""
  ...

```

Test Cases

```

solve(1000) # => "1"
solve(150) # => "110"
solve(147) # => "1100"

```

Clever's Spec

```

def problem_spec
-- function signature

```

```

(implementation: Nat → String)
-- inputs
(n: Nat) :=
-- spec
let spec (result : String) :=
  0 < n →
  result.all (fun c => c = '0' ∨ c = '1') →
  Nat.ofDigits 2 (result.data.map (fun c => if c = '0' then 0 else 1)).reverse =
  ↪ (Nat.digits 10 n).sum
-- program termination
∃ result,
  implementation n = result ∧
  spec result

```

Observation

The post-condition appears to be underspecified.

A potential concern is that `result.all (fun c => c = '0' ∨ c = '1')` appears as the antecedent of an implication rather than as an independent conjunct. As a result, the specification only requires the decoded value to equal the digit sum *when* all characters happen to be '0' or '1' — it does not independently enforce that the result consists solely of binary digits. An implementation may return a string containing arbitrary characters and still satisfy the specification, because the implication is vacuously true whenever the antecedent is false. For example:

```

n      = 1000
expected = "1"      # intended output (digit sum of 1000 is 1, binary "1")
result   = "xyz"     # also satisfies the spec (antecedent is false, implication
↪ holds vacuously)

```

The intended condition should conjoin both requirements:

```

0 < n →
result.all (fun c => c = '0' ∨ c = '1') ∧
Nat.ofDigits 2 (result.data.map (fun c => if c = '0' then 0 else 1)).reverse =
↪ (Nat.digits 10 n).sum

```

Problem 88

Problem Description

Description

```

def sort_array(lst : list(int)) -> list(int):
    """
    Given an array of non-negative integers, return a copy of the given array
    ↪ after sorting,
    you will sort the given array in ascending order if the sum( first index
    ↪ value, last index value) is odd,
    or sort it in descending order if the sum( first index value, last index
    ↪ value) is even.
    Note(George): I have elected to ignore the copy part."""
    ...

```

Test Cases

```
sort_array([])          # => []
sort_array([5])         # => [5]
sort_array([2, 4, 3, 0, 1, 5]) # => [0, 1, 2, 3, 4, 5]
sort_array([2, 4, 3, 0, 1, 5, 6]) # => [6, 5, 4, 3, 2, 1, 0]
```

Clever's Spec

```
def problem_spec
  -- function signature
  (implementation: List Nat → List Nat)
  -- inputs
  (lst: List Nat) :=
  -- spec
  let spec (result : List Nat) :=
    lst.length > 0 →
    result.length = lst.length ∧
    (∀ i, i < result.length →
      result[i]! ∈ lst ∧
      lst[i]! ∈ result ∧
      result.count (lst[i]!) = lst.count (lst[i]!)) ∧
    (lst.head! + lst.getLast!) ≡ 1 [MOD 2] →
    result.Sorted Nat.le ∧
    (lst.head! + lst.getLast!) ≡ 0 [MOD 2] →
    result.Sorted (fun a b => a ≥ b)
  -- program termination
  ∃ result,
    implementation lst = result ∧
    spec result
```

Observation

The post-condition appears to be underspecified.

A potential concern is that the specification opens with `lst.length > 0 →`, treating the non-emptiness of the input as a precondition. When `lst` is empty, this precondition is false and the entire implication is vacuously satisfied, meaning any result — including `[1, 2, 3]` — would be accepted. However, the test cases explicitly include the empty input with the expected output `[]`:

```
sort_array([]) # => []
```

This suggests that the empty input is an intended case and should be constrained by the specification. The current formulation effectively leaves the output for empty input unconstrained, which may allow implementations that return a non-empty list for empty input to pass verification. We suggest either removing the `lst.length > 0 →` guard and handling the empty case explicitly within the spec, or adding a separate conjunct such as `lst.length = 0 → result = []` to cover this case.

Problem 89

Problem Description

Description

```
def encrypt(str : str) -> str:
    """Create a function encrypt that takes a string as an argument and
    returns a string encrypted with the alphabet being rotated.
    The alphabet should be rotated in a manner such that the letters
    shift down by two multiplied to two places."""
    ...
```

Test Cases

```
encrypt("hi")           # => "lm"
encrypt("asdfghjkl")    # => "ewh,jklnop"
encrypt("gf")            # => "kj"
encrypt("et")            # => "ix"
```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: String → String)
-- inputs
(str: String) :=
-- spec
let spec (result : String) :=
    result.data.all (fun c => c.isLower) →
    result.length = str.length ∧
    (∀ i, i < str.length →
        let c := str.data[i]!
        let c' := result.data[i]!
        ((c'.toNat - 97) + 2 * 2) % 26 = (c.toNat - 97))
-- program termination
∃ result,
    implementation str = result ∧
    spec result
```

Observation

In the postcondition, `result.data.all (fun c => c.isLower)` appears as the antecedent of an implication rather than as an independent conjunct. As a result, any implementation that returns a string containing non-lowercase characters would make the antecedent false and thereby vacuously satisfy the entire postcondition, without being required to satisfy the length or character-mapping constraints. It may be worth considering whether this condition should instead be connected with $\wedge$, so that the result is always required to consist of lowercase letters and to meet the remaining constraints simultaneously.

Problem 94

Problem Description

Description

```
def sum_largest_prime(lst : list[int]) -> int:
    """You are given a list of integers.
    You need to find the largest prime value and return the sum of its digits."""
    ...
```

Test Cases

```
sum_largest_prime([0,3,2,1,3,5,7,4,5,5,5,2,181,32,4,32,3,2,32,324,4,3]) # => 10
sum_largest_prime([1,0,1,8,2,4597,2,1,3,40,1,2,1,2,4,2,5,1]) # => 25
sum_largest_prime([1,3,1,32,5107,34,83278,109,163,23,2323,32,30,1,9,3]) # =>
↳ 13
sum_largest_prime([0,724,32,71,99,32,6,0,5,91,83,0,5,6]) # =>
↳ 11
sum_largest_prime([0,81,12,3,1,21]) # =>
↳ 3
sum_largest_prime([0,8,1,2,1,7]) # =>
↳ 7
```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: List Nat → Nat)
-- inputs
(lst: List Nat) :=
-- spec
let spec (result : Nat) :=
  lst.any (fun num => Nat.Prime num) →
  result > 0 ∧ ∃ i, i < lst.length ∧ Prime (lst.get! i) ∧
  (∀ j, j < lst.length ∧ Prime (lst.get! j) → lst.get! i ≤ lst.get! j) ∧
  result = (Nat.digits 10 (lst.get! i)).sum
-- program termination
∃ result,
  implementation lst = result ∧
  spec result
```

Observation

The ordering constraint $\forall j, j < \text{lst.length} \wedge \text{Prime} (\text{lst.get! } j) \rightarrow \text{lst.get! } i \leq \text{lst.get! } j$ requires that the element at index i is less than or equal to every other prime in the list, which appears to characterize the *smallest* prime rather than the largest. It may be worth considering whether the inequality should be reversed to $\text{lst.get! } j \leq \text{lst.get! } i$ to better reflect the intended semantics.

Problem 96

Problem Description

Description

```
def count_up_to(n : int) -> list[int]:
    """Implement a function that takes a non-negative integer and returns an
    ↪ array of the first n
    integers that are prime numbers and less than n."""
    ...
```

Test Cases

```
count_up_to(5)    # => [2,3]
count_up_to(11)   # => [2,3,5,7]
count_up_to(0)    # => []
count_up_to(20)   # => [2,3,5,7,11,13,17,19]
count_up_to(1)    # => []
count_up_to(18)   # => [2,3,5,7,11,13,17]
```

Clever's Spec

```
def problem_spec
-- function signature
(implementation: Nat → List Nat)
-- inputs
(n: Nat) :=
-- spec
let spec (result : List Nat) :=
  match n with
  | 0 => result = []
  | n => n > 0 → (∀ i, i < result.length → (Nat.Prime (result.get! i)) ∧
    ↪ (result.get! i) < n) ∧
    (∀ i : Nat, i < n → Nat.Prime i → i ∈ result)
-- program termination
∃ result,
  implementation n = result ∧
  spec result
```

Observation

The postcondition specifies that every element in the result is prime and less than n , and that every prime less than n appears in the result, but it does not constrain the order of elements. This appears to be at odds with the name `count_up_to` and the test cases, which all produce primes in ascending order. An implementation returning primes in an arbitrary or descending order would satisfy the current spec. It may be worth adding an ordering constraint, e.g., $\forall i, j, i < j \rightarrow j < \text{result.length} \rightarrow \text{result.get! } i < \text{result.get! } j$.

Problem 156

Problem Description

Description

```
def int_to_mini_roman(num: Nat) -> String:
    """Given a positive integer, obtain its roman numeral equivalent as a string,
    and return it in lowercase.
    Restrictions: 1 <= num <= 1000"""
    ...
```

Test Cases

```
int_to_mini_roman(19)  # => "xix"
int_to_mini_roman(152) # => "clii"
int_to_mini_roman(426) # => "cdxxvi"
```

Clever's Spec

```
def problem_spec
-- function signature
(impl: Nat → String)
-- inputs
(num: Nat) :=
-- spec
let spec (result: String) :=
1 ≤ num ∧ num ≤ 1000 ∧ (result.data.all (fun c => c.isLower)) →
isValidRoman result ∧ romanToDecimal result = num
-- program terminates
∃ result, impl num = result ∧
-- return value satisfies spec
spec result
```

Observation

In the postcondition, `result.data.all (fun c => c.isLower)` appears as part of the antecedent of an implication rather than as an independent conjunct. As a result, any implementation returning a string that contains non-lowercase characters (e.g., "XIX") would make the antecedent false and thereby vacuously satisfy the spec, without being required to produce a valid Roman numeral or a correct value. It may be worth considering whether this condition should be connected with `∧` instead, so that the result is always required to consist of lowercase letters alongside the other constraints.

Problem 161

Problem Description

Description

```
def solve(string : String) -> String:
    """You are given a string s.
    if s[i] is a letter, reverse its case from lower to upper or vice versa,
    otherwise keep it as it is.
    If the string contains no letters, reverse the string.
    The function should return the resulted string."""
    ...
```

Test Cases

```
solve("1234") # => "4321"
solve("ab")   # => "AB"
solve("#a@c") # => "#A@c"
```

Clever's Spec

```
def problem_spec
-- function signature
(impl: String → String)
-- inputs
(string : String) :=
-- spec
let spec (result: String) :=
result.length = string.length ∧
let hasNoAlphabet := string.all (λ c => not (c.isAlpha));
(hasNoAlphabet →
  result.toList = string.toList.reverse) ∧
(hasNoAlphabet = false →
  ∀ i, i < string.length →
  let c := string.get! <i>;
  (c.isAlpha → ((c.isLower → c.toUpper = result.get! <i>) ∨
    (c.isUpper → c.toLower = result.get! <i>))) ∧
  (¬ c.isAlpha → c = result.get! <i>))
-- program terminates
∃ result, impl string = result ∧
-- return value satisfies spec
spec result
```

Observation

In the branch handling strings that contain alphabetic characters, the case analysis for each letter uses $\vee$ to connect the two implications:

```
(c.isLower → c.toUpper = result.get! <i>) ∨
(c.isUpper → c.toLower = result.get! <i>)
```

Since a character cannot be simultaneously lowercase and uppercase, for any alphabetic character one of the two antecedents is always false, making the corresponding implication vacuously true and therefore the entire disjunction trivially satisfied. As a result, the spec places no actual constraint on how alphabetic characters appear in the output. It may be worth considering replacing $\vee$ with $\wedge$, so that the relevant implication is properly enforced depending on the case of the character.